

UNIVERSIDADE ESTADUAL DO MARANHÃO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

SISTEMA DE CORREÇÃO AUTOMÁTICA DE GABARITOS POR IMAGEM

Disciplina: Visão Computacional

Professor: Lucio Campos

Aluno: Luis Guilherme Busaglo Lopes

Instituição: UEMA

Sumário

1	Introdução	2
2	Objetivos	2
2.1	Objetivo geral	2
2.2	Objetivos específicos	2
3	Tecnologias utilizadas	3
4	Funcionamento do sistema	3
5	Processamento de imagem	3
6	Banco de dados	4
7	Interface web	5
8	Deploy	5
9	Resultados esperados	5
10	Conclusão	6
11	Referências	6

1 Introdução

O presente projeto foi desenvolvido no contexto da disciplina de Visão Computacional, com o objetivo de aplicar técnicas de processamento de imagens em um problema prático: a leitura e correção automática de gabaritos de múltipla escolha. A proposta consiste em permitir que uma imagem do gabarito seja enviada ao sistema, processada por algoritmos de visão computacional e registrada em uma base de dados com os resultados da correção.

O sistema foi implementado em Python, utilizando a biblioteca OpenCV para o processamento das imagens, Streamlit para a interface web e uma camada de persistência preparada para utilizar SQLite em ambiente local ou PostgreSQL/Supabase em ambiente de deploy. Com isso, o projeto pode ser executado tanto no computador do desenvolvedor quanto em uma aplicação hospedada na nuvem.

2 Objetivos

2.1 Objetivo geral

Desenvolver uma aplicação capaz de identificar, processar e corrigir gabaritos a partir de imagens, utilizando técnicas de visão computacional e registrando os resultados em banco de dados.

2.2 Objetivos específicos

- Capturar ou receber imagens de gabaritos por meio de uma interface web.
- Detectar a área do gabarito na imagem enviada.
- Aplicar transformações e limiarização para facilitar a identificação das marcações.
- Comparar as respostas detectadas com um conjunto de respostas corretas.
- Calcular acertos, erros e pontuação final.
- Armazenar os dados da correção em banco local ou remoto.
- Permitir consulta posterior dos registros por meio da interface do sistema.

3 Tecnologias utilizadas

O projeto utiliza as seguintes tecnologias principais:

- **Python**: linguagem principal da aplicação.
- **OpenCV**: biblioteca responsável pelo processamento de imagens.
- **NumPy**: manipulação de matrizes e dados numéricos das imagens.
- **Pandas**: exibição e exportação de dados em formato tabular.
- **Streamlit**: construção da interface web.
- **SQLite**: armazenamento local dos resultados.
- **Supabase/PostgreSQL**: armazenamento remoto utilizado no deploy.

4 Funcionamento do sistema

O fluxo principal do sistema começa com o acesso do usuário à interface web. Na tela de correção, o usuário informa ou confirma o ID do candidato e envia uma imagem do gabarito. O sistema também possui suporte para captura por câmera do navegador, embora em celulares o envio de imagem seja mais confiável por causa das restrições de permissão em conexões HTTP.

Após o recebimento da imagem, o arquivo é decodificado em uma matriz de pixels. Em seguida, o OpenCV executa o pipeline de análise, que inclui a detecção do gabarito, o ajuste da perspectiva, a binarização da imagem e a leitura das regiões correspondentes às alternativas. As respostas identificadas são comparadas com o gabarito oficial configurado no sistema.

Ao final da análise, o sistema apresenta a imagem processada, o recorte do gabarito, a imagem limiarizada, a tabela de respostas e a pontuação obtida. Os dados são gravados no banco, junto com caminhos das imagens de apoio e informações como data, hora, ID do candidato e hash da foto.

5 Processamento de imagem

O processamento da imagem é a parte central do projeto. O sistema trabalha com a ideia de localizar o gabarito na foto e transformar essa região em uma imagem mais

adequada para leitura automática. Para isso, são aplicadas etapas comuns em problemas de visão computacional:

- Conversão e preparação da imagem recebida.
- Identificação da região correspondente ao gabarito.
- Correção de perspectiva para reduzir distorções.
- Limiarização, separando regiões marcadas e não marcadas.
- Análise das posições das alternativas com base em coordenadas previamente definidas.
- Classificação das respostas como corretas, erradas, vazias ou múltiplas.

Essa abordagem permite que a aplicação lide com imagens capturadas por celular, desde que a foto esteja em boas condições de iluminação, enquadramento e nitidez.

6 Banco de dados

O sistema utiliza uma tabela chamada `capturas`, responsável por armazenar os registros das correções. Entre os principais campos salvos estão:

- ID interno do registro.
- ID do candidato.
- Data e hora da captura.
- Status geral da correção.
- Respostas detectadas.
- Quantidade de acertos e erros.
- Pontuação final.
- Caminhos das imagens geradas.
- Hash da foto, usado para evitar duplicidade.

Durante o desenvolvimento local, o banco pode ser armazenado em SQLite. Para deploy, o projeto foi adaptado para utilizar Supabase, que fornece uma base PostgreSQL remota. Essa adaptação permite que os dados continuem disponíveis mesmo quando a aplicação estiver hospedada fora do computador do desenvolvedor.

7 Interface web

A interface foi desenvolvida com Streamlit. Ela possui três abas principais:

- **Correção por foto:** área onde o usuário envia a imagem do gabarito e executa a correção.
- **Planilhas:** área para visualização de arquivos CSV ou XLSX presentes no projeto.
- **Banco:** área para consulta dos registros salvos, com filtros por ID e período.

Uma melhoria implementada no sistema foi a sugestão automática do próximo ID de candidato. O sistema consulta o banco, identifica o maior ID numérico já cadastrado e preenche automaticamente o próximo valor. Após uma nova foto ser salva, o campo é atualizado para receber a próxima captura.

8 Deploy

Para tornar o sistema independente da execução local no computador, o projeto foi preparado para deploy em ambiente web. O Streamlit Cloud pode executar o arquivo principal `p1_lucio/app_streamlit.py`. As dependências são instaladas por meio do arquivo `requirements.txt`, e a conexão com o Supabase é configurada por variável de ambiente.

No ambiente de deploy, a variável `SUPABASE_DB_URL` armazena a connection string do banco PostgreSQL. Essa informação não deve ser enviada ao GitHub, pois contém credenciais sensíveis. Ela deve ser configurada apenas no painel de secrets da plataforma de hospedagem.

9 Resultados esperados

Espera-se que o sistema consiga corrigir gabaritos simples a partir de imagens com boa qualidade. O resultado apresentado ao usuário inclui a pontuação, a relação de respostas identificadas e as imagens auxiliares do processamento. Além disso, os dados ficam salvos no banco para consulta posterior.

O projeto demonstra a aplicação prática de conceitos de visão computacional, como segmentação, detecção de regiões, limiarização e análise de formas. Também integra esses conceitos a uma aplicação web funcional, com persistência de dados e possibilidade de deploy.

10 Conclusão

O desenvolvimento do sistema permitiu aplicar conceitos de Visão Computacional em um problema real de automação acadêmica. A leitura de gabaritos por imagem exige cuidados com captura, iluminação, perspectiva e contraste, fatores que influenciam diretamente na qualidade da detecção.

A solução desenvolvida integra processamento de imagem, interface web e banco de dados, formando um fluxo completo desde o envio da foto até o armazenamento do resultado. Com a adaptação para Supabase e Streamlit Cloud, o projeto também passa a funcionar de forma independente do computador local, tornando-se acessível pela internet.

11 Referências

- BRADSKI, Gary. The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000.
- OPENCV. OpenCV Documentation. Disponível em: <https://docs.opencv.org/>.
- STREAMLIT. Streamlit Documentation. Disponível em: <https://docs.streamlit.io/>.
- SUPABASE. Supabase Documentation. Disponível em: <https://supabase.com/docs>.